🗂️ Complete Backend Structure

```
spvs-backend/
│
├── server.js                    ← Entry point, starts Express server
├── package.json                 ← Dependencies list
├── .env                         ← Secret keys (never commit this)
├── .env.example                 ← Template showing what .env needs
│
├── config/
│   ├── db.js                    ← MongoDB connection
│   └── cloudinary.js            ← Cloudinary (image uploads)
│
├── models/                      ← MongoDB schemas (database tables)
│   ├── Admin.js
│   ├── Blog.js
│   ├── Faculty.js
│   ├── Alumni.js
│   ├── Gallery.js
│   ├── Announcement.js
│   ├── Enquiry.js
│   ├── Download.js
│   ├── Result.js
```

```
│   ├── Testimonial.js
│   ├── Job.js                    ← NEW (needs to be added)
│   ├── JobApplication.js         ← NEW (resume upload + apply form)
│   ├── CertificateRequest.js     ← NEW (student cert requests)
│   └── SiteStatus.js             ← NEW (page live/maintenance status)
│
├── controllers/                  ← Business logic for each feature
│   ├── authController.js
│   ├── blogController.js
│   ├── facultyController.js
│   ├── alumniController.js       ← NEW (currently in otherControllers)
│   ├── galleryController.js
│   ├── announcementController.js
│   ├── enquiryController.js
│   ├── downloadController.js     ← NEW
│   ├── resultController.js       ← NEW
│   ├── testimonialController.js  ← NEW
│   ├── jobController.js          ← NEW
│   ├── jobApplicationController.js ← NEW
│   ├── certificateController.js  ← NEW
│   └── siteStatusController.js   ← NEW
│
```

```
├── routes/                    ← URL paths mapped to controllers
│   ├── authRoutes.js
│   ├── blogRoutes.js
│   ├── facultyRoutes.js
│   ├── alumniRoutes.js        ← NEW
│   ├── galleryRoutes.js
│   ├── announcementRoutes.js  ← NEW
│   ├── enquiryRoutes.js       ← NEW
│   ├── downloadRoutes.js      ← NEW
│   ├── resultRoutes.js        ← NEW
│   ├── testimonialRoutes.js   ← NEW
│   ├── jobRoutes.js           ← NEW
│   ├── jobApplicationRoutes.js ← NEW
│   ├── certificateRoutes.js   ← NEW
│   └── siteStatusRoutes.js    ← NEW
│
├── middleware/
│   ├── auth.js                ← JWT token check (protects admin routes)
│   ├── upload.js              ← Multer (handles file uploads)
│   └── errorHandler.js        ← Catches all errors in one place
│
└── utils/
```

└── seed.js ← Seeds dummy data into MongoDB for testing

🔍 How Each Part Works

server.js

The main file. It starts Express, connects to MongoDB, registers all routes, and starts listening on a port. Every request to the backend comes in here first.

config/db.js

Connects to your MongoDB database using the URL from .env. Called once when server starts.

config/cloudinary.js

Sets up Cloudinary credentials. Used when admin uploads a faculty photo, alumni photo, blog image, or gallery image. Image goes to Cloudinary → Cloudinary gives back a URL → that URL is saved in MongoDB.

Models (Database Tables)

Model	What it stores
Admin.js	Admin username + hashed password
Blog.js	Title, content, image, category, date, featured flag
Faculty.js	Name, role, dept, qualification, experience, photo, status
Alumni.js	Name, batch, field, role, photo, featured flag
Gallery.js	Image URL, title, category, date

Model	What it stores
Announcement.js	Title, date, link, active/inactive
Enquiry.js	Name, phone, email, message, type, read/unread status
Download.js	File name, category, file URL, date
Result.js	Student name, reg no, class, marks, pass/fail
Testimonial.js	Parent/student name, review text, rating, approved flag
Job.js	Title, dept, type, qualification, experience, open/closed
JobApplication.js	Applicant name, phone, email, resume URL, job ID, status
CertificateRequest.js	Student name, reg no, DOB, cert type, status, admin note
SiteStatus.js	Page name, path, status (live/maintenance/offline/coming_soon)

Controllers (Logic)

Each controller has these standard functions:

Auth Controller

- login → checks username/password → returns JWT token → frontend stores it in localStorage

Blog Controller

- getAllBlogs → returns all blogs (public)
- getBlogById → returns one blog (public)

- createBlog → admin creates new blog (protected)
- updateBlog → admin edits blog (protected)
- deleteBlog → admin deletes (protected)

Faculty Controller — same CRUD pattern, plus image upload via Cloudinary

Alumni Controller — same CRUD, plus photo upload, plus toggle featured

Gallery Controller — upload image to Cloudinary, save URL, delete

Enquiry Controller

- createEnquiry → anyone from website submits contact form → saved in DB
- getAllEnquiries → admin sees all enquiries (protected)
- markRead → admin marks as read (protected)
- deleteEnquiry → admin deletes (protected)

Job Controller — admin manages job openings (CRUD)

JobApplication Controller

- applyForJob → anyone submits apply form with resume → resume uploaded to Cloudinary → saved in DB
- getAllApplications → admin sees all applicants (protected)
- updateStatus → admin marks Shortlisted / Rejected / Hired (protected)

CertificateRequest Controller

- submitRequest → student fills request form on website → saved in DB
- getAllRequests → admin sees all requests (protected)

- updateStatus → admin approves/rejects + adds note (protected)

SiteStatus Controller

- getStatus → returns all pages and their status (public — so frontend can show maintenance page)
- updatePageStatus → admin changes a page status (protected)
- updateSiteMode → admin sets whole site to maintenance/offline (protected)

Middleware

auth.js — runs before every protected route. Reads the JWT token from the request header, verifies it, if valid lets the request through, if invalid returns 401 Unauthorized.

upload.js — uses Multer library. When a file is uploaded (photo/resume/PDF), Multer catches it from the form data before it reaches the controller. Then the controller sends it to Cloudinary.

errorHandler.js — wraps the whole app. If any controller throws an error, this catches it and sends a clean JSON error response instead of crashing the server.